

Assignment 3

Virtual Cameras and Projections

Computer Graphics Course
C++ | CLion | MiniFB | MicroUI | GLM

Student 1 Natalie Bittar ID:212586044

Student 2 Adan Awad ID:32430267

Course Computer Graphics Assignment Basic Graphics and Immediate Mode GUI

This report documents the implementation and testing of coordinate frames, virtual cameras, view and projection matrices, geometric normals, LookAt, and Dolly Zoom.

Report Structure

- Part 1: Coordinate Frames and Bounding Boxes
- Part 2: Virtual Camera and View Matrix
- Part 3: Perspective and Orthographic Projection
- Part 4: Face and Vertex Normals
- Part 5: Pair Programming Extensions - LookAt and Dolly Zoom
- Interaction and Testing
- Conclusion

Overview

Assignment 3 extends the wireframe viewer from Assignment 2 by introducing a complete camera pipeline. Instead of projecting model coordinates directly to the framebuffer, every vertex now passes through Model, View, and Projection transformations before the perspective divide and viewport mapping.

```
clip_position = Projection * View * Model * vertex;
```

The application continues to support multiple OBJ meshes and independent model transformations. The new controls allow the user to inspect local and world coordinate frames, switch projection modes, move and rotate the camera, visualize bounding boxes and normals, and experiment with LookAt and Dolly Zoom camera behavior.

Part 1: Coordinate Frames and Bounding Boxes

World and Local Coordinate Axes

Two types of coordinate axes were implemented as visual debugging helpers. World axes are fixed at the world origin (0,0,0), while local axes are transformed with the active model. The conventional color mapping is used: red for the X axis, green for the Y axis, and blue for the Z axis.

```
X axis: MFB_RGB(255, 60, 60)
Y axis: MFB_RGB(60, 255, 80)
Z axis: MFB_RGB(80, 130, 255)
```

The local axes are generated by transforming the local origin and three short axis endpoints with the model local/world matrix. This keeps the local frame attached to the model when it is translated or rotated. The world axes are projected directly from fixed world-space coordinates.

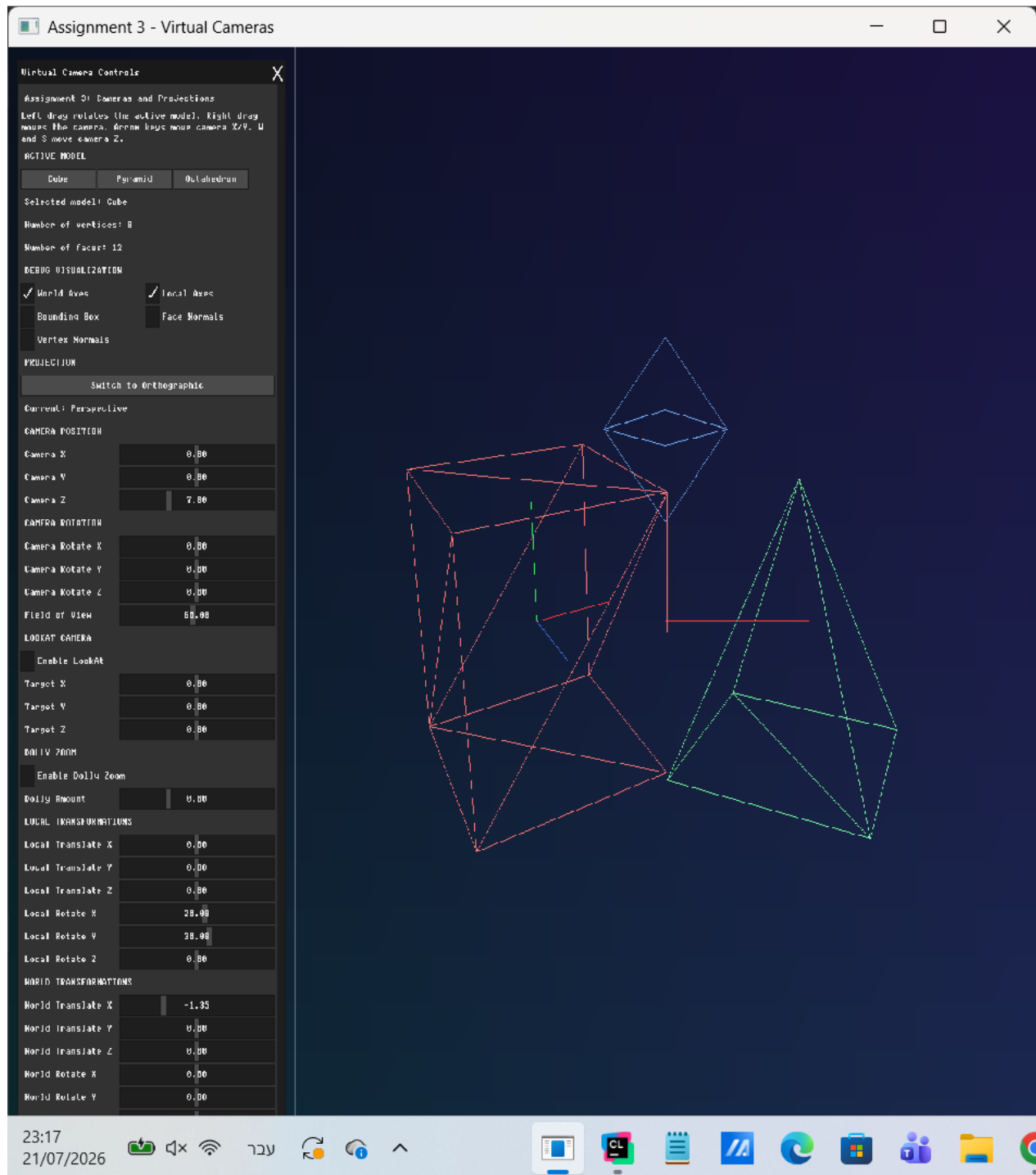


Figure 1. Main scene in perspective mode with world and local coordinate axes

Three-Dimensional Bounding Box

The axis-aligned bounding box is calculated from the minimum and maximum x, y, and z coordinates of the original mesh. Eight 3D corners are created from all combinations of minimum and maximum coordinates, transformed by the model matrix, and connected with twelve edges.

```
std::array<glm::vec3, 8> corners = {
    {min.x, min.y, min.z}, {max.x, min.y, min.z},
    {max.x, max.y, min.z}, {min.x, max.y, min.z},
    {min.x, min.y, max.z}, {max.x, min.y, max.z},
```

```

    {max.x, max.y, max.z}, {min.x, max.y, max.z}
};

```

Because the corners are transformed by the same model matrix as the mesh, the yellow wireframe box follows the selected model during translation, rotation, and scaling.

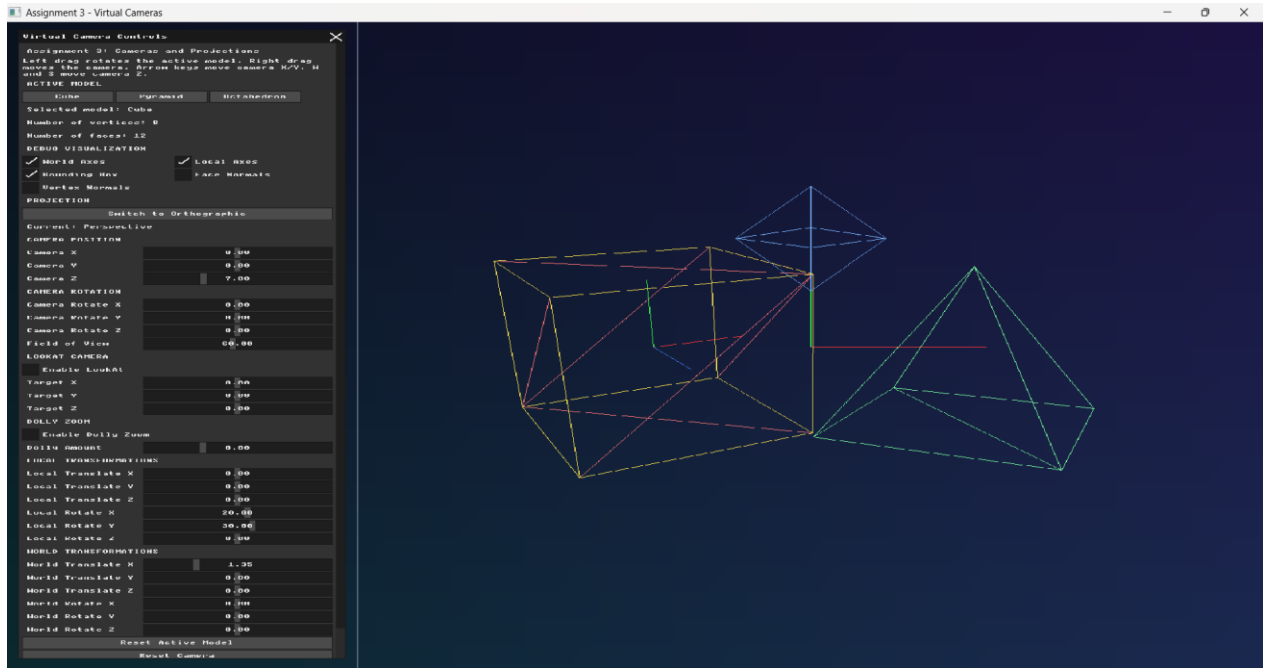


Figure 2. Bounding box of the active model

Part 2: The Virtual Camera and View Matrix

Camera State

A Camera structure stores position, Euler rotation, LookAt target, field of view, clipping planes, projection mode, and Dolly Zoom state.

```

struct Camera {
    glm::vec3 position{0.0f, 0.0f, 7.0f};
    glm::vec3 rotation{0.0f};
    glm::vec3 target{0.0f};
    float field_of_view = 60.0f;
    float near_plane = 0.1f;
    float far_plane = 100.0f;
};

```

Sliders expose the camera X, Y, Z position and X, Y, Z rotation. Keyboard controls and right-mouse dragging also modify the camera state directly.

Constructing the View Matrix

The camera is represented mathematically by a transform in world space. Rendering requires the inverse transform because the camera illusion is produced by moving the entire world in the opposite direction. The program creates the camera transform and uses `glm::inverse` to produce the View matrix.

```

camera_transform = translate(I, camera.position) * rotation;
View = inverse(camera_transform);

```

The final order for each vertex is Model first, then View, then Projection:

```
clip = P * V * M * vec4(vertex, 1.0f);
```

This behavior was verified by changing Camera X: moving the camera left causes the scene to appear to move right, which is the expected inverse relationship.

Part 3: Perspective and Orthographic Projection

Perspective Projection

Perspective projection was implemented using `glm::perspective`. The projection depends on the vertical field of view, framebuffer aspect ratio, and near/far clipping planes.

```
projection = glm::perspective(  
    glm::radians(camera.field_of_view),  
    aspect_ratio,  
    camera.near_plane,  
    camera.far_plane  
);
```

After multiplying by the projection matrix, homogeneous clip coordinates are converted to normalized device coordinates by dividing x, y, and z by w. This perspective divide makes distant geometry appear smaller.

```
ndc = vec3(clip) / clip.w;
```

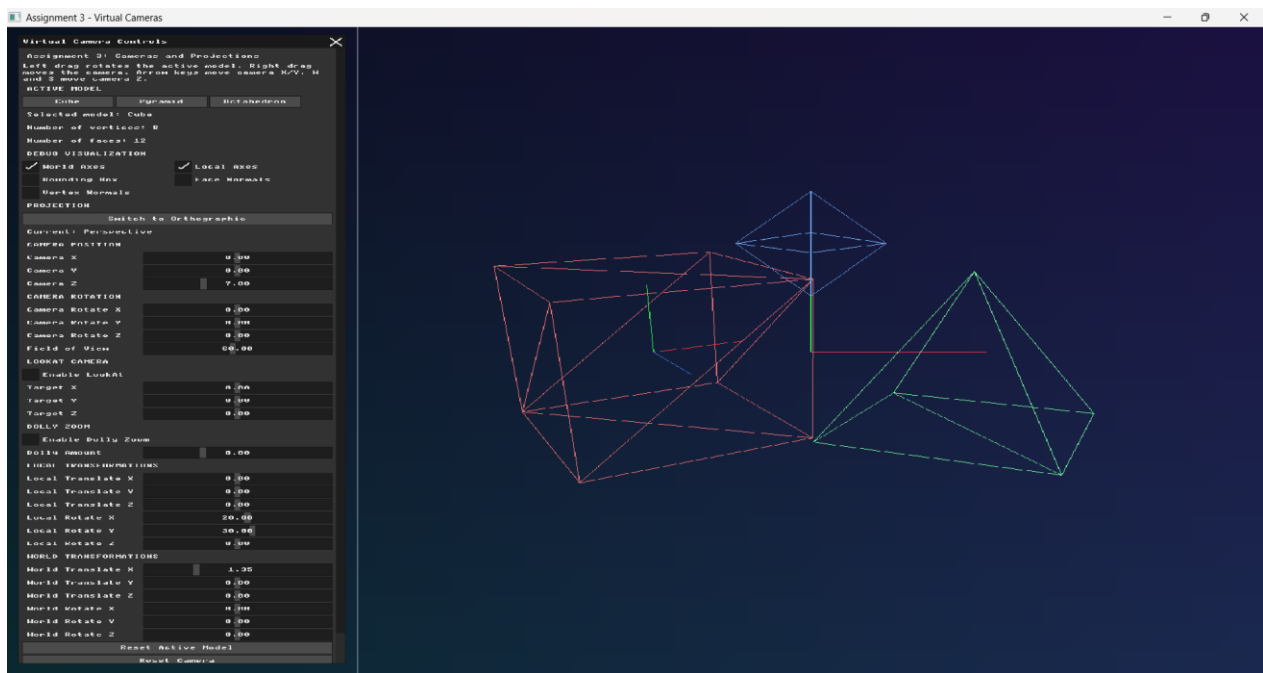


Figure 3. Scene rendered using perspective projection

Orthographic Projection

Orthographic mode uses `glm::ortho` and preserves object size regardless of depth. A GUI button toggles between projection matrices while keeping the same scene and camera state, making the visual difference easy to compare.

```
projection = glm::ortho(  
    -ortho_width, ortho_width,  
    -ortho_height, ortho_height,
```

```

    near_plane, far_plane
);

```

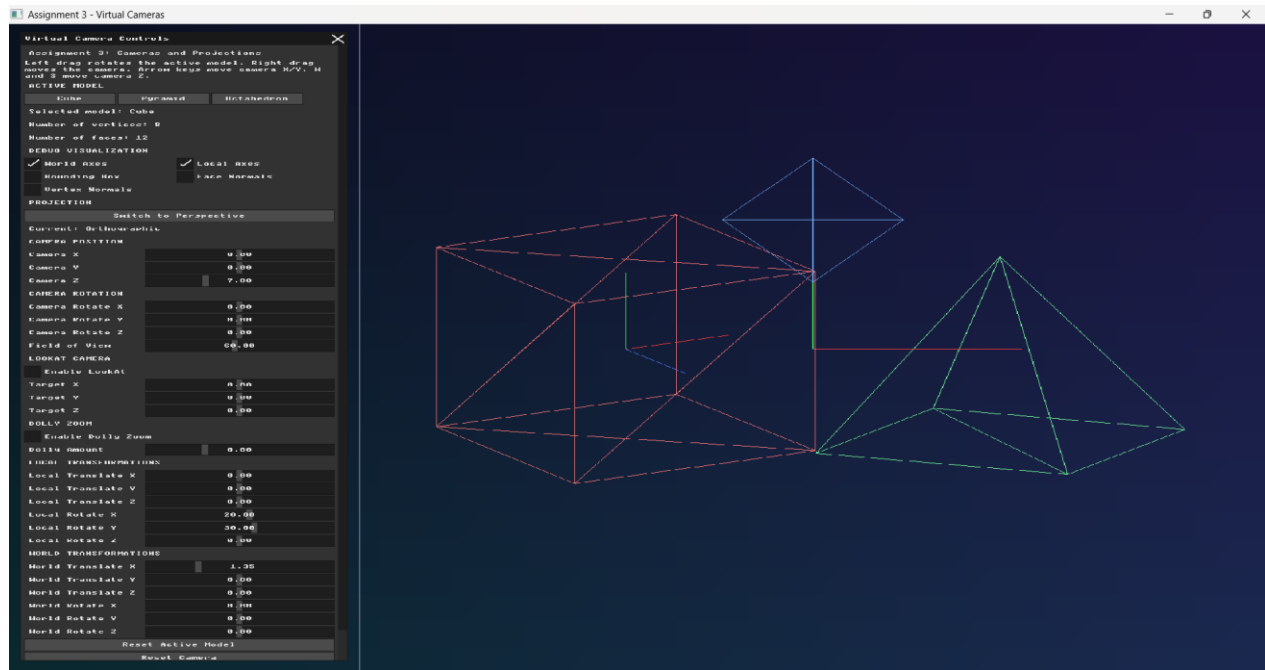


Figure 4. Scene rendered using orthographic projection

Projection comparison: In perspective mode, depth affects apparent size and parallel lines may converge. In orthographic mode, objects retain the same scale at different depths and the scene looks flatter.

Part 4: Calculating and Visualizing Normals

Face Normals

For each triangular face, two edge vectors are created. Their cross product gives a vector perpendicular to the triangle. The vector is normalized to unit length and stored as the face normal.

```

edge_ab = b - a;
edge_ac = c - a;
face_normal = normalize(cross(edge_ab, edge_ac));

```

Face normals are drawn from the center of each triangle. The center is the average of the three triangle vertices. In the visualization, face normals are magenta.

Vertex Normals

Every vertex normal is calculated by summing all face normals from faces that share the vertex and normalizing the final result. This produces an averaged direction suitable for smooth shading in future assignments.

```

vertex_normals[face.a] += face_normal;
vertex_normals[face.b] += face_normal;
vertex_normals[face.c] += face_normal;

```

Normals must transform correctly under non-uniform scale. Therefore, the implementation uses the inverse-transpose normal matrix derived from the model matrix rather than multiplying normals directly by the model matrix.

```
normal_matrix = transpose(inverse(mat3(model_matrix)));
```

Vertex normals are drawn in cyan, while face normals are drawn in magenta. Both remain correctly attached to the model as it rotates.

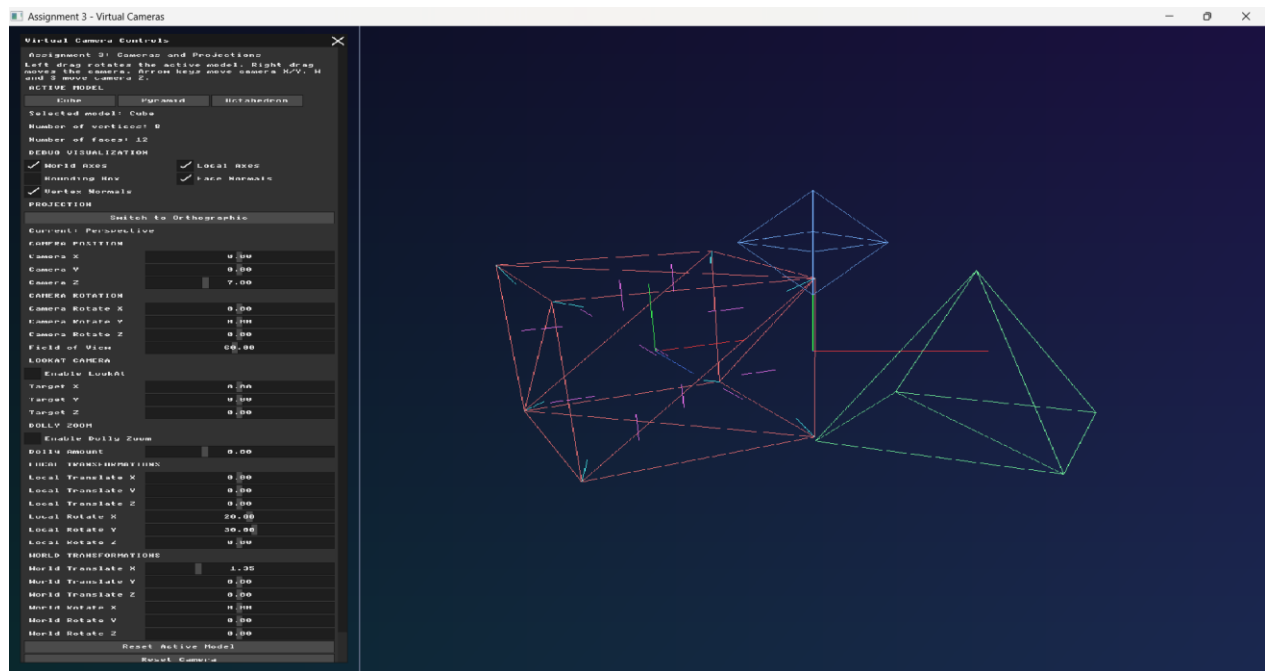


Figure 5. Face normals and vertex normals of the active model

Part 5: Pair Programming Extensions

LookAt Camera Mode

LookAt mode constructs the View matrix from three vectors: the camera position, a target point, and the world up vector. This avoids manually finding camera rotation angles.

```
View = glm::lookAt(
    camera.position,
    camera.target,
    glm::vec3(0.0f, 1.0f, 0.0f)
);
```

The GUI provides Target X, Target Y, and Target Z sliders. When LookAt is enabled, the camera remains directed toward the target even when its position changes. A safety check handles the degenerate case where the camera position and target are almost identical.

Dolly Zoom (Vertigo Effect)

The Dolly Zoom control links camera distance and field of view. Moving the camera farther away normally makes the subject smaller, but simultaneously reducing the FOV can compensate for that size change. The implementation preserves the projected size approximately by keeping the following quantity constant:

$$\text{distance} * \tan(\text{FOV} / 2) = \text{constant}$$

For a new camera distance d , the corresponding FOV is computed as:

$$\text{newFov} = 2 * \text{atan}(\text{constant} / d)$$

The Dolly Amount slider updates Camera Z and FOV together. The active subject remains approximately stationary in size while the perspective relationship between objects changes, creating the characteristic Vertigo effect.

Interaction and Testing

The final application was built and run successfully through the CMake target. The following controls and behaviors were tested:

- World Axes and Local Axes checkboxes toggle the colored coordinate frames.
- Bounding Box displays a yellow wireframe box around the active model.
- Face Normals and Vertex Normals display correctly and rotate with the model.
- The projection button switches between Perspective and Orthographic modes.
- Camera position and rotation sliders modify the View matrix in real time.
- Left mouse drag rotates the active model.
- Right mouse drag moves the camera.
- Arrow keys move the camera along X and Y; W and S move it along Z.
- LookAt uses the camera position and target coordinates to build the View matrix.
- Dolly Zoom links camera Z and field of view.
- Cube, Pyramid, and Octahedron remain independently transformable.

Conclusion

Assignment 3 transformed the previous wireframe renderer into a camera-based 3D viewing system. The complete rendering pipeline now uses Model, View, and Projection matrices, followed by the perspective divide and viewport conversion. Debug axes, bounding boxes, and geometric normals make the invisible 3D coordinate systems visible and easier to verify.

The assignment also demonstrates the practical difference between orthographic and perspective projections, the inverse nature of camera motion, the importance of the inverse-transpose matrix for normals, and the construction of LookAt and Dolly Zoom camera effects. These components provide the mathematical foundation required for later lighting and shading assignments.